



WAF MINI HACKATHON 2026

Build Lightweight Security Gateway for Real-World Abuse Protection

OFFICIAL COMPETITION RULES

April – May 2026

Company-wide Competition — Open to All Tech Staff

~400 Engineers · Minimum 3 members per team

1ST PLACE: \$30,000 | 2ND PLACE: \$20,000 | 3RD PLACE: \$10,000

3x ENCOURAGEMENT PRIZE: \$5,000

1. COMPETITION OBJECTIVES

WAF Mini Hackathon 2026 is a large-scale internal tech competition open to all Engineering staff, with the following strategic goals:

- Build a battle-ready Internal WAF + Anti-Abuse Gateway capable of protecting company systems against real-world threats (bots, fraud, DDoS, relay attacks...).
- Identify, develop, and recognise Security Engineering talent and Future Tech Leaders within the company.
- Produce a core security platform that can be deployed to production immediately after the competition.

2. ELIGIBILITY & SCALE

Parameter	Details
Participants	All Tech staff
Format	Team-based competition
Team size	Minimum 3 members — no maximum cap
Required language	Rust for the Core component (mandatory — no exceptions), while the control plane, dashboard, etc can use Node.js or any other language
Competition period	April – May 2026

3. COMPETITION SCHEDULE

Phase	Timeline	Key Activities
Kick-off & Training	Week 1-2	Official rules announced · Team registration · Sandbox environment setup
Development Sprint	Weeks 3-4-5	System build ·
Hardening & Testing	Week 6	Optimise & test · Red Team Dry Run · Code Freeze · Official submission
Final & Attack Battle	Week 7	10-min demo per team · 45-min Attack Battle · Real-time scoring · Award ceremony

4. COMPETITION MISSION

Each team must build a complete Mini WAF (Web Application Firewall) / Security Gateway that sits in front of and protects the ENTIRE target website — not just a handful of specific endpoints. The WAF operates as a full reverse proxy: every request from the internet passes through the WAF before reaching the backend, and every response from the backend passes through the WAF before being returned to the client.

Why: Attackers don't only hit /login — they scan the entire site, probing /admin, /api, static assets that may leak information, and then exploit what they find. A WAF that only protects a fixed list of endpoints will be bypassed through other routes.*

Deployment Architecture

Internet → [WAF / Security Gateway] → Backend Application

The WAF must be transparent to the backend — the backend should require no knowledge of the WAF's existence and no code changes.

Tiered Protection Policy

The WAF applies different protection policies to different route groups, from most strict to baseline:

Tier	Route pattern	Policy applied	Notes
CRITICAL	/login /otp /deposit /withdrawal	Full stack: per-user rate limit, device fingerprint, behavioural check, transaction velocity, fail-close, canary	Fail-close on WAF error — deny all
HIGH	/game/* /api/* /user/*	DDoS protection, per-IP & per-session rate limit, OWASP detection, smart caching, bot filter	Fail-open for normal traffic
MEDIUM	/static/* /assets/* /public/*	Basic rate limit, path traversal detection, aggressive caching	High throughput — latency priority
CATCH-ALL	All remaining routes /**	Baseline: SQLi/XSS detect, rate limit, block known-bad IPs, log everything	No route left unprotected

All traffic — regardless of tier — must be: inspected inbound & outbound, written to the audit log, given a risk score, and subject to the global blacklist.

The system must automatically detect and defend against any attack the Red Team launches against any route on the website.

5. TECHNICAL REQUIREMENTS

5.1 Core Constraints (MANDATORY)

Language: Rust (strictly mandatory — no exceptions)

Output: Single binary — no runtime dependencies, no Docker required

Start command:

```
./waf run
```

Mode: Reverse Proxy — bidirectional HTTP/HTTPS inspection (inbound & outbound)

Performance SLA: p99 latency overhead <= 5ms | throughput >= 5,000 req/s baseline

5.2 Core Features (MANDATORY)

#	Feature	Technical description
01	Rule Engine	Match on IP, Path, Header, Payload, Cookie — supports regex, wildcard, exact match, logical AND/OR
02	Rate Limiting (per-user)	Sliding window per IP AND per user-session — not just per IP. Token bucket for burst control
03	DDoS Protection	Burst detection + auto block + configurable threshold per route tier. Fail-close for CRITICAL tier, fail-open for MEDIUM/CATCH-ALL
04	Challenge Engine	JS Challenge, Proof-of-Work. Adaptive: Allow / Challenge / Block based on accumulated risk score
05	Relay & Proxy Detection	Proxy chain detection, X-Forwarded-For chain validation, abnormal header patterns, ASN classification (residential vs datacenter vs Tor)
06	Whitelist + Blacklist	IP & FQDN whitelist. Threat intel blacklist loaded from file at startup (Tor exit list, bad ASN). Auto risk boost for known-bad IPs
07	Smart Caching	No caching for CRITICAL tier routes. Aggressive caching for MEDIUM tier (/static, /assets). Configurable TTL per route pattern
08	Device Fingerprinting	Generate persistent device ID from: TLS fingerprint (JA3/JA4), HTTP/2 settings, User-Agent entropy, Accept-Encoding pattern. Detect same device rotating IPs to bypass blocks
09	Behavioural Anomaly Detection	Detect: too-uniform request timing (bot), zero-depth session (hitting CRITICAL route without passing through homepage), missing Referer on sensitive routes, inter-request interval < 50ms
10	Transaction Velocity & Sequence	Per-user cross-route tracking: Login→OTP→Deposit within N seconds. Withdrawal velocity check after deposit. Rapid limit-change pattern detection

5.3 Attack Detection Coverage (MANDATORY — OWASP Top 5 minimum)

Note: The original spec mentioned only SQLi. A production WAF must cover all attack vectors below. This is the minimum requirement for the system to qualify as a real WAF.

Attack Vector	Detection requirement
SQL Injection (SQLi)	Classic, blind, time-based, UNION-based. Detect in URL params, headers, JSON body
Cross-Site Scripting (XSS)	Reflected & stored XSS payload detection. Script injection in query strings, form data, JSON
Path Traversal	../ sequences, URL-encoded variants (%2e%2e), detect in URL path & query params
SSRF Attempt	Requests to internal IP ranges (10.x, 172.16.x, 192.168.x, 169.254.x), metadata endpoints
HTTP Header Injection	Host header injection, response splitting (CRLF injection), X-Forwarded-For spoofing

Attack Vector	Detection requirement
Brute Force / Credential Stuffing	Per-user failed login counter, password spraying pattern (many different users from same IP)
Error Scanning / Recon	Rapid 4xx/5xx pattern, endpoint enumeration, OPTIONS method abuse
Request Body Abuse	Malformed JSON, oversized payload (> configurable limit), deeply nested objects, content-type mismatch

5.4 Flexible Rule System (MANDATORY)

- Add/edit/delete rules WITHOUT rebuilding the binary — hot-reload is mandatory
- Rule format: YAML or TOML. Each rule must define: condition (match), action (allow/block/challenge/rate-limit), risk_score_delta
- Rule scope: global (entire website), per-tier (CRITICAL/HIGH/MEDIUM/CATCH-ALL), per-route-pattern, per-IP, per-user-session, per-device-fingerprint
- Rule priority: numeric priority for resolving conflicts when multiple rules match

5.5 Challenge & Risk Engine (MANDATORY)

- Risk score accumulates per {IP + device fingerprint + session} — does not reset after each request
- Risk score increases on: rule match, failed challenge, behavioural anomaly, suspicious ASN, device fingerprint conflict
- Risk score decreases on: successful challenge, sustained normal behaviour over a time window
- Decision thresholds are configurable: score < 30 = Allow, 30–70 = Challenge, > 70 = Block
- Canary Endpoints / Honeypots: deploy decoy paths (/admin-test, /api-debug). Any request that hits one = auto set risk score to MAX, block IP immediately

5.6 Realtime Dashboard (MANDATORY)

- Live feed: real-time request log with request ID, timestamp (ms), risk score, action taken, rule triggered
- Attack visualisation: attack type distribution chart, top attacker IPs, route heatmap
- Hot config: update rules, toggle actions (block/challenge/allow), adjust thresholds — NO service restart required
- Structured audit log: JSON format, append-only, SIEM-ingestible. Each entry: request_id, ts_ms, ip, device_fp, risk_score, rule_id, action

5.7 Response Filtering & Outbound Protection (MANDATORY)

Company-specific requirement: the WAF must inspect responses before returning them to the client, preventing accidental leakage of internal information.

- Block responses containing stack traces, internal IPs, API keys, or verbose error messages (5xx with body exceeding a configurable size)
- Mask/redact sensitive fields in JSON responses (configurable field list: card_number, bank_account, ...)
- Detect and block accidental PII leakage in response headers (X-Debug, X-Internal-*)

5.8 Graceful Degradation (MANDATORY)

The Attack Battle will include scenarios where the WAF itself is DDoS'd. Behaviour under stress must be explicitly defined.

- Fail-close mode for the CRITICAL tier: reject all traffic if a WAF internal error occurs — safer than allowing pass-through

- Fail-open mode for MEDIUM & CATCH-ALL tiers: allow traffic through if the WAF is overloaded, log a warning for later review
- Fail-close or fail-open must be configurable per route tier in the rule file — never hardcoded
- Circuit breaker for the upstream backend: if the backend is unresponsive, the WAF returns 503 rather than hanging connections

5.9 Advanced Features (BONUS — extra points)

Feature	Difficulty	Description
HTTPS / TLS termination	Medium	TLS termination & mTLS support, configurable cipher suites
Geographic Restriction	Medium	GeoIP lookup (MaxMind lite DB), block/challenge from restricted jurisdictions, VPN geo-bypass detection
IP Reputation Feed	Low	Load Tor exit list + bad ASN list from file at startup, periodic refresh. Auto risk boost
Multi-region Deployment	High	waf deploy --region=sg,eu,us. Config sync across regions
Zero-downtime Config Sync	High	Rolling config update with no service downtime, config versioning
Auto Scaling	High	Horizontal scaling with shared state via Redis/etc
Behavioural ML Scoring	Very high	Lightweight ML model to classify bot vs human based on request sequence patterns

6. SCORING CRITERIA (Total: 120 points)

Criterion	Evaluation details	Points
Security Effectiveness	Detect rate & false positive rate under real Attack Battle. OWASP Top 5 coverage: SQLi, XSS, Path Traversal, SSRF, Header Injection. Device fingerprinting accuracy. Behavioural anomaly detection. Canary endpoint functionality.	40 / 120
Performance	p99 latency overhead <= 5ms. Throughput >= 5,000 req/s. Memory footprint. Behaviour under DDoS load.	20 / 120
Intelligence & Adaptiveness	Risk score accuracy (per user + device + session). Transaction velocity & sequence detection. Graceful degradation under overload. Correct fail-close/fail-open behaviour per tier.	20 / 120
Architecture & Code Quality	Rust code quality, idiomatic patterns, error handling, documentation, test coverage	15 / 120

Criterion	Evaluation details	Points
Extensibility	Rule system: hot-reload, per-scope (IP/user/session/device/tier), priority resolution. Plugin-ready architecture. Configurable response filtering.	10 / 120
Dashboard UI/UX & Realtime Config	Live request feed with structured log. Attack visualisation. Hot config update. JSON audit log.	10 / 120
Deployment & Operability	Single binary, one-command startup, documented fail behaviour, upstream circuit breaker	5 / 120

Note: Security Effectiveness accounts for 33% of the total score — it is the most important criterion, evaluated holistically across OWASP coverage and company-specific threats (device fingerprinting, behavioural anomaly, transaction sequence).

7. ATTACK BATTLE

Attack Battle Procedure

- The Red Team is designated by the organising committee — comprising Security Leads and external security experts.
- Each team faces a 45-minute live attack. The Red Team may attack ANY route on the website — not only sensitive endpoints.**
- The system must AUTOMATICALLY detect and defend. Manual intervention is STRICTLY PROHIBITED.**
- Judges monitor each team's realtime dashboard throughout the Attack Battle.
- The Red Team will include a scenario that DDoS-attacks the WAF itself — graceful degradation behaviour will be scored.**

Attack Scenarios (extended)

#	Attack Vector	Techniques
01	DDoS Layer 4 & 7	TCP/UDP flooding, HTTP flood, Slowloris, RUDY. Includes DDoS targeting the WAF itself to test graceful degradation
02	Bot Login & Credential Stuffing	Brute force login, password spraying across multiple IPs, distributed credential stuffing with IP rotation
03	Relay & Proxy Attack	Proxy chain injection, abnormal X-Forwarded-For, VPN/Tor exit nodes, datacenter IP → /login attack
04	Device Fingerprint Evasion	Rotate TLS fingerprint, cycle User-Agent, spoof residential IP while using the same underlying device for multi-accounting
05	Behavioural Bypass	Zero-depth session attack (direct hit to CRITICAL route), perfectly-timed bot requests, spoofed Referer headers
06	Transaction Fraud Pattern	Login→Deposit in < 5s, withdrawal immediately after deposit, rapid limit-change + withdrawal pattern
07	OWASP Injection	SQLi variants (blind, time-based), XSS payload in JSON body, SSRF to internal IP, path traversal

#	Attack Vector	Techniques
08	Canary / Recon Scan	Endpoint enumeration, honeypot endpoint probing, OPTIONS method abuse, error response harvesting

8. PRIZES & REWARDS

Official prizes

Rank	Cash prize
 1st place	\$30,000 USD
 2nd place	\$20,000 USD
 3rd place	\$10,000 USD
 Consolation place x3	\$5,000 USD

9. MANDATORY RULES & PROHIBITIONS

Mandatory

- The WAF must sit in front of and protect the ENTIRE website — it may not proxy only a selected subset of routes while ignoring the rest.
- The CRITICAL tier (login, otp, deposit, withdrawal) must receive the strictest protection with all policies fully implemented as specified.
- Cross-route abnormal patterns must be detected: Login → Deposit in rapid succession, multi-IP/session, zero-depth session into CRITICAL routes.
- The demo must use real traffic with the WAF running live in front of the full backend — hardcoded mock responses are not permitted.
- The system must defend itself autonomously during the Attack Battle — no one may manually intervene.

Prohibited

- ✗ Using fake data or a staged demo — IMMEDIATE DISQUALIFICATION.
- ✗ Hardcoding rules solely to pass specific test cases — IMMEDIATE DISQUALIFICATION.
- ✗ Manual intervention in the system during the Attack Battle — IMMEDIATE DISQUALIFICATION.
- ✗ Attacking another team's sandbox environment during the Development phase.

Violation of any prohibition results in immediate disqualification without further explanation.

10. REGISTRATION & PROCESS

Step	Action	Details
01	Individual registration	Complete the internal form: name, level, primary stack, preferred role. Deadline: end of Week 2.
02	Form a team	Self-organise or be matched by the organising committee based on skill set. Minimum 3 members per team.
03	Submit team details	Team Lead submits: team name, member list. Organising committee confirms and grants sandbox access.
04	Development & check-ins	Build the system.
05	Code Freeze & submission	End of Week 5: submit repo + binary (amd64 Linux) + README. No commits after freeze.
06	Pre-final verification	Organising committee verifies: ./waf run executes, connects to test target, dashboard is accessible.
07	Final Demo + Attack Battle	10-min demo per team. 45-min Attack Battle. Scored against the official rubric.
08	Awards & post-hackathon	Awards ceremony. Top 3 teams onboard the Core Security Team. HR processes promotions within 2 weeks.

Judging panel

- CEO
- CTO
- Tech Manager
- Head of Security
- Red Team (scores the Attack Battle)